

Objective

The goal of this project is to **design, implement, and analyze** a parallel high-performance simulation of the **Convolution Operation program**. Starting from the sequential implementation of the problem, students will parallelize it using three different parallel programming paradigms:

- **OpenMP** for shared-memory parallelism,
- **Hybrid (MPI-OpenMP)** for distributed-memory parallelism, and
- **CUDA** for GPU-based parallelism.

With each implementation, students will analyze the impact of different parallelization strategies on performance, scalability and implementation complexity.

Description of the Problem

Convolution [1] is a *mathematical* operation on two functions **f** and **g** used to produce a third function that is interpreted as a modified version of one of the original functions. Basically, it gives the area overlapped between both functions as a function of the amount that one of the original functions is translated, as it is shown in Figure 1.

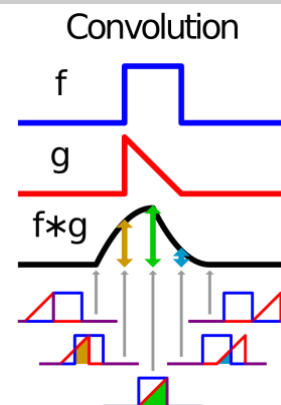


Figure 1 Convolution process

Convolution is used on probability, statistics, computer vision, natural language processing, image and signal processing, engineering, and differential equations.

To clarify the objectives of the convolution process, it will be presented applied in the field of image processing. In this field, the convolution process is widely used to change the intensity of each individual pixel to reflect the intensities of the surrounding area.

A common use of convolution is to create image filters as could be Blur, Sharpen, Emboss and Edge detection. Figure 2 shows the results after applying the emboss filter.



Figure 2 Emboss effect

The convolution process in the field of image processing is based on applying a

matrix named as “convolution kernel”. This matrix consists of an $n \times n$ matrix identifying the surrounding area of the evaluated pixel. Each cell of the kernel contains a pixel weight in respect of the pixel.

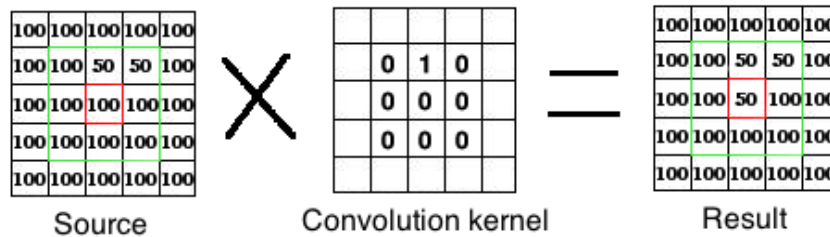


Figure 3 3x3 Convolution Kernel. The red cell represents the evaluated pixel.

In the example shown in Figure 3, the convolution process corresponds to the individual multiplication of every paired cells/pixels, from the source and the kernel. Finally, it obtains the sum of all of them, which corresponds to the resulting pixel value, as it is presented in the Figure 4.

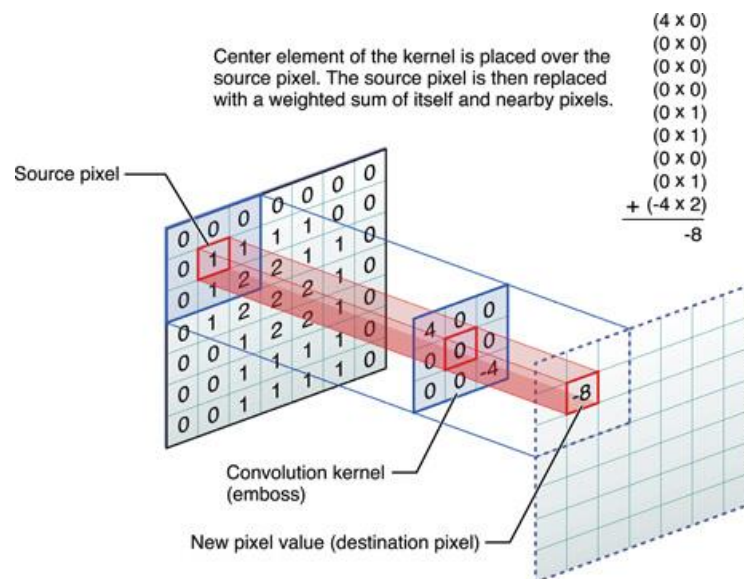


Figure 4 Convolution process

Varying the composition of the kernel, it is possible to obtain different results. Figure 5 shows different image convolution results.





Figure 5 Convolution operations with different kernel matrix.

It is also possible to have more complex functions than the simple *add* to achieve different results.

The convolution process complexity depends on the original matrix size, the kernel dimension and on the mathematical function applied to the cells. By this reason, its parallelization is an important technique to consider in further implementations.

Serial Code

To compare the performance and the results obtained from different groups solutions, we will provide the test-bed that you must pass for providing your results and elaborate the reports. The material you have for this project is as follows:

- **Serial code:** Source code for a serial implementation. This serial implementation allows you to process small and large images. For large images (>500MB) you will have problems allocating enough memory for processing convolution. If you want to test for images bigger than the images provided for us, then you would need to split the original images into partitions. The serial code provided, the number of partitions is an argument of the program.
- **PPM images:** In folder “/share/apps/files/convolution/images”, located into each node of the cluster moore, you can find the set of images to test. Don’t copy these images into your \$HOME folder given that disk space at the front-end is scarce and will be exhausted very soon.
- **Kernel matrix:** In folder “/share/apps/files/convolution/kernel” of the cluster *moore*, you can find the set of kernel matrices. Kernel matrices are described using text files. The kernel provided 5x5 *Sharpen* is very useful in image processing area; the big ones (25x25, 49x49, 99x99) are randomly generated for testing the scalability of your proposals.
- The serial code generates a text file with the time spent in different execution phases. The Figure 6 shows an example of the output of the serial convolution program.

```

[sisco@moore CONVOLUTION]$ more CONVOLUTION.o42954
Imatge: /share/apps/files/convolution/images/im03.ppm
ISizeX : 800
ISizeY : 600
kSizeX : 99
kSizeY : 99
0.177648 seconds elapsed for Reading image file.
0.004053 seconds elapsed for copying image structure.
0.072701 seconds elapsed for Reading kernel matrix.
45.625081 seconds elapsed for make the convolution.
0.093309 seconds elapsed for writing the resulting image.
46.097404 seconds elapsed
todas las tareas han terminado
[sisco@moore CONVOLUTION]$ |

```

Figure 6. Screenshot of the serial convolution output.

In addition, the serial code generates a PPM image as the result of the convolution between the original image and the selected kernel. You may store the resulting images in your \$HOME directory; however, your disk quota may be exceeded quickly. Therefore, we strongly recommend transferring these files to your laptop using the sftp command and then deleting them from the cluster.

Source code compilation and execution

- For the serial source code compilation, you should execute as follows the Figure 7.

```

$gcc convolution.c -o serialconv
$./serialconv
share/apps/files/convolution/image/source_image.ppm
share/apps/files/convolution/kernel/kernel.txt
output_image.ppm number_partitions

```

Figure 7. Arguments of the serial convolution program.

Remember that is **totally forbidden to execute the code in the front-end** of the cluster. Doing this, you can collapse the server or produce a server downfall. By this, it is important to use the front-end queuing system even for serial programs.

For executing the serial code into the *moore cluster*, you can use the same script used with the serial NAS parallel benchmarks together with the *qsub command*. See Figure 8.

```

$qsub run-simple-serial.sh

```

Figure 8. Execute the serial code into the cluster.

To have serial execution times as a reference for future parallelization versions, you should calculate the execution time of the serial convolution program for arguments given on Table 1.

Image	Kernel			
	kernel5x5_S harpen.txt	Kernel25x25_R andom2.txt	Kernel49x49_R andom2txt	Kernel99x99- Random2.txt
im01.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im02.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im03.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im04.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im05.ppm	Partitions=3	Partitions=3	Partitions=3	Partitions=3

Table 1. Table with arguments to calculate the elapsed time times of the serialconv.

Note that the combination of ima05.ppm with any kernel must be executed with at least three partitions. Also, keep in mind that serial execution with the im05.ppm image and the 49x49 and kernel99x99 kernels can take over 2000s and 10000s, respectively.

Project Statement: Hybrid and CUDA parallelization practice

Delivery 2: Implementation with MPI-OpenMP (Deadline 10th of May)

In this activity, we should take as starting point the previous OpenMP implementation. Thus, we will focus on the implementation using the MPI programming model. It means that we should analyse which is the best way to decompose the data between nodes. You can choose to develop one of the following two different versions, which work as follows:

a) *Static mapping of tasks*

It consists in dividing the region into a fixed number of fragments, which are processed in parallel by different processors/cores. Note that the allocation is done at the beginning of the execution of the program and cannot be modified. It can be classified into quadrants, rows, columns, etc.

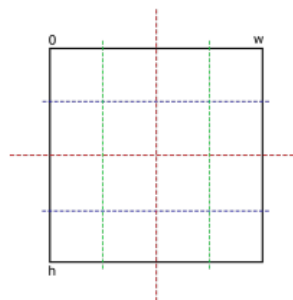


Figure 9. Static decomposition.

b) Dynamic mapping of tasks

In this case, the fragments will be mapped to different processors/cores as soon as they finish with the previous calculations.

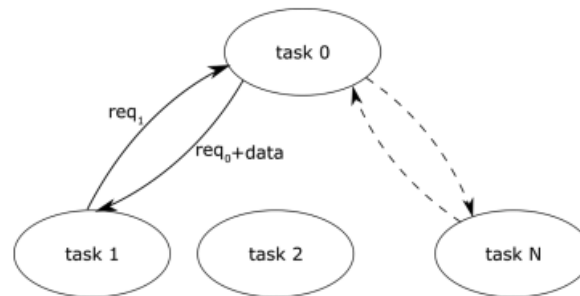


Figure 9. Dynamic decomposition.

For the chosen strategy, you must analyse the performance of each implementation at the following points:

- Speedup compared to the number of cores.
- Identify if there is imbalanced.
- Quantify the additional cost (overhead) of parallelization.

You can optionally develop both strategies. Obviously, we will assess it positively. In such a case, explain the strengths and weaknesses of each implementation and discuss some scenarios where you think it would be more convenient to use and why.

To measure the time in the MPI programs, we recommend using the `MPI_Wtime` function, as the flowing example shows:

```

float x, y, z;
double start, elapsed;
start = MPI_Wtime();
z=x*y;
elapsed = MPI_Wtime() - start;
  
```

To execute the MPI parallel version or the Hybrid parallel version, you can use the scripts located in “/share/apps/files/convolution/scripts” folder of the moore cluster.

In your report, you should compare the performance of the hybrid solution in relation to the OpenMP solution implemented in the previous delivery for different problem’s size and cores/nodes.

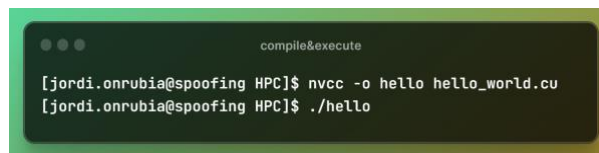
Remember that this delivery has a weight of 35% over the total mark of the subject.

Delivery 3: Implementation with CUDA (Deadline 8th of June)

In this activity, you should propose a new implementation using CUDA C++, you should try to use different Grid and Thread sizes configurations as well as different image sizes. Compare the performance and scalability of the problem, and compare it as far as possible, with the results obtained in the previous deliveries.

At the end of the document, write a brief explanation about CPU computing vs GPU. This explanation must be about your experience, how hard is the programming, cost-benefit...

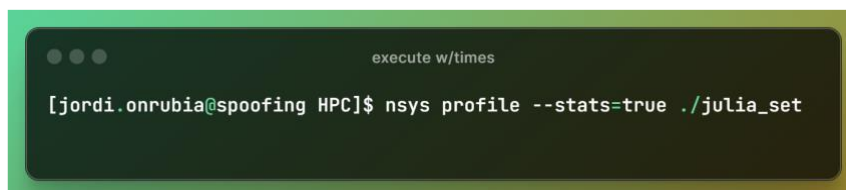
The Figure 10 shows as to compile and execute a CUDA program.



```
compile&execute
[jordi.onrubia@spoofing HPC]$ nvcc -o hello hello_world.cu
[jordi.onrubia@spoofing HPC]$ ./hello
```

Figure 10. How to compile and execute a CUDA program.

The Figure 11 shows as to get the execution time.



```
execute w/times
[jordi.onrubia@spoofing HPC]$ nsys profile --stats=true ./julia_set
```

Figure 11. How to get the execution time.

You can also use the classic way of computing the $time = end_time - start_time$, with the time library.

This implementation will be carried out in the GPU server: *spoofing-gcd.udl.cat*.

During the next weeks, a user account for each student will be created in the *spoofing-gcd GPU server*.

The deadline to submit this implementation is the 8th of June (CUDA Program).

Remember that this delivery has a weight of 30% over the total mark of the subject.

Delivery Instructions

For both implementations, students must deliver the source code. The code will be delivered using a *github repository*. The code should contain explanations of the main decisions and library functions used to parallelize the problem. A **github** reference to the code must be included in the corresponding report.

For each implementation, you should also do a brief PDF report (maximum 10 pages with Hybrid and 8 pages with CUDA) with the following items:

- I. **Main decisions taken to parallelize the problem:** You should explain how the problem was parallelized from its original serial version, considering, in the case of the Hybrid practice, both architectures: shared memory using OpenMP and distributed memory using MPI. Provide a clear justification for the overall parallelization strategy adopted at the distributed-memory level. If multiple strategies were tested, briefly describe each of them and explain the rationale behind your final choice.
- II. **Scalability of the program:** You should show the *Execution Time*, together with the *Speedup and Efficiency* for different number of processes/cores and problem size (image and kernel). It is better if you use figures instead of tables.
- III. **Discussion of the obtained results:** You should explain the results for the main metrics (*Speedup and Efficiency*) and the reasons for the obtained results. In the case of the Hybrid practice, try to compare it in relation to the OpenMP results. Consider in your discussion the effects of the dependences, synchronization and memory traffic between data, in case it was needed, in your solution. In the case of CUDA, you should compare it in relation to the Hybrid practice.
- IV. **Conclusions:** A brief paragraph that concisely outlines the problem addressed, highlights the most significant results, and provides a critical reflection on both the findings and the parallelization method employed.

If you need more pages, you can put the least significant results into annexes.

You should deliver reports throughout the Sakai virtual campus, by means of the corresponding *Activity Section*. **All the activities can be done individually or by groups of two students.**

To make reports understandable and easily evaluated or reviewed by any reader, you must follow the rules of good practices to prepare a report. Each report must contain the title of the activity and the authors. Use the editing techniques properly: tables, figures, cross-references, table of contents, bibliography etc.

Each report should be complete, justifying the design decisions, making a proper analysis of the performance and the comparison of the effects of different alternatives to reach the more appropriate decision. You can insert some code pieces for helping to explain your solutions. The figures must be correctly formatted considering aspects such as titles, axis labels, legend, type of lines, symbols, colours, etc. The text in the figures, labels and axis must be legible. The axis must be correctly dimensioned. The type of chart must be selected correctly depending on that you want to explain. Figures must be accompanied by a brief explanation.

Bibliography

1.- Gonzalez, R. C. and Woods, R. E. *Digital Image Processing*. Prentice Hall, 2008.